

Concise Report

Company Name: _____

Website Link: http://127.0.0.1:8080

Date: 2024-10-15

Time: 15:17:04

Risk Summary

High Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
SQL Injection	1	(9.1%)	The description of the vulnerability is: SQL Injection Vulnerability. This vulnerability can occur when user input is not properly validated or sanitized on the server side, allowing an attacker to execute arbitrary SQL code. The impact of this vulnerability can range from data corruption to unauthorized access to sensitive data. It is important to apply the principle of least privilege and avoid using the'sa' or 'db-owner' database users to minimize the impact of this vulnerability.	Use a vetted library or framework that does not allow this weakness to occur. Use structured mechanisms that automatically enforce the separation between data and code. Process queries using prepared statements, parameterized queries, or stored procedures. Do not dynamically construct and execute query strings using "exec" or similar.

Medium Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
--------------------	-------	---------	-------------	------------

Absence of Anti-CSRF Tokens	4	(36.4%)	Based on the info: 'Summary No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy. Solution Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes the ability to detect and prevent session fixation attacks, which can be used in conjunction with CSRF attacks to gain unauthorized access to a user's session. In summary, cross-site request forgery (CSRF) is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. CSRF attacks are effective in situations where the victim has an active session on the target site, is authenticated via HTTP auth on the target site, or is on the same local network as the target site. To prevent CSRF attacks, use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Use anti-CSRF packages such as the OWASP CSRFGuard. Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Use the ESAPI Session Management control to detect and prevent session fixation attacks, which can be used in conjunction with CSRF attacks to gain unauthorized access to a user's session.	Most CSRF defenses can be bypassed using attacker-controlled script. Use anti-CSRF packages such as the OWASP CSRFGuard. When the user performs a dangerous operation, send a separate confirmation request. Use the "double-submitted cookie" method as described by Felten and Zeller.
-----------------------------	---	---------	---	---

Content Security Policy (CSP) Header Not Set		(36.4%)	The vulnerability can be described as a missing Content Security Policy (CSP) header and an obsolete Content Security Policy (CSP) header being found in the system. The first vulnerability (10038-1) refers to a missing Content Security Policy (CSP) header, while the second vulnerability (10038-2) refers to an obsolete Content Security Policy (CSP) header. The third vulnerability (10038-3) indicates the presence of a report-only header within the Content Security Policy (CSP). To address this vulnerability, it is crucial to set the Content Security Policy (CSP) headers correctly and ensure that they are up-to-date and relevant to the application's requirements. This can be achieved by reviewing and modifying the existing Content Security Policy (CSP) settings or by implementing a new one if necessary. Additionally, removing any report-only headers from the Content Security Policy (CSP) is essential to mitigate the third vulnerability.	X-Frame- options allows developers of web content to restrict the usage of their application within the form of overlays, frames, or iFrames. The developer can indicate from which domains can frame the content. Implementation of this protection mechanism is in development to cover gaps. There is a need for allowing frames from multiple domains. improper usage of frames in web applications.
Missing Anti-clickjacking Header	4	(36.4%)	The vulnerability can be described as a non-compliant configuration of the X-Frame-Options header in an application, which may lead to clickjacking attacks. The Anti-clickjacking header is missing, and there are multiple entries for the X-Frame-Options header, some of which have malformed code. This can potentially allow an attacker to bypass the clickjacking protection provided by the header.	Leverage the HttpOnly flag when setting a sensitive cookie in a response. Attackers might also be able to use XMLHttpRequest to read the headers directly and obtain the cookie. While this mitigation is effective for protecting cookies from a browser's own scripting engine, third-party components or plugins may have their own engines that allow access to cookies.

Low Risk Vulnerabilities

Vulnerability Name	Count	Count %	Description	Mitigation
Cookie No HttpOnly Flag	1	(9.1%)	The vulnerability is that a cookie can be accessed by JavaScript if it is not set with the HttpOnly flag. This can lead to the cookie being transmitted to another site and potentially allowing for session hijacking if the cookie is a session cookie.	Set the SameSite attribute of a sensitive cookie to 'Lax' or 'Strict' This instructs the browser to apply this cookie only to same-domain requests. This provides a good Defense in Depth against CSRF attacks. While this mitigation is effective for protecting cookies from a browser's own scripting engine, third-party components or plugins may have their own engines that allow access to cookies.

Cookie without SameSite Attribute	1	(9.1%)	The vulnerability can be described as follows: Affected Software: ZAP Proxy with CookieSameSiteScanRule enabled Vulnerability Overview: The CookieSameSiteScanRule scans for cookies that lack the SameSite attribute and cookies that have an invalid SameSite attribute. The lack of the SameSite attribute can lead to cross-site scripting (XSS) attacks, while an invalid SameSite attribute can result in arbitrary code execution. Description of the Vulnerabilities: 1. Vulnerability 10054-1: Cookie without SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that do not have the SameSite attribute. Attackers can exploit this vulnerability by setting cookies without the SameSite attribute, which can lead to cross-site scripting (XSS) attacks. 2. Vulnerability 10054-2: Cookie with SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that have the SameSite attribute. There is no known vulnerability associated with this scenario. 3. Vulnerability 10054-3: Cookie with Invalid SameSite Attribute This vulnerability occurs when the CookieSameSiteScanRule scans for cookies that have an invalid SameSite attribute. Attackers can exploit this vulnerability by setting cookies with an invalid SameSite attribute, which can result in arbitrary code execution. Mitigation: Disable the CookieSame	Consider using language-theoretic security (LangSec) techniques that characterize inputs using a formal language. Use an input validation framework such as Struts or the OWASP ESAPI Validation API. Explicitly set the encoding you are using whenever the protocol allows you to.
X-Content-Type-Options Header Missing9		(81.8%)	The vulnerability allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.	